

Lab Sheet 2 — Setup & Core Networking Concepts

Development Environment & Core Networking Concepts in Practice

Course	977-337 Network Programming and Simulation
Student	Hadis Rodpradit
Student ID	6630613039
Program	Digital Engineering (International), College of Computing, PSU Phuket
Environment	Ubuntu 24.04.4 LTS (noble) · Python 3.12.3 · venv: netprog-env

This report documents completion of the six lab tasks for Lab Sheet 2, followed by the verification checklist and reflection answers. All scripts were run inside the netprog-env virtual environment; each task is evidenced by a terminal / browser screenshot.

Task 1 (Part A) — Virtual environment & repository

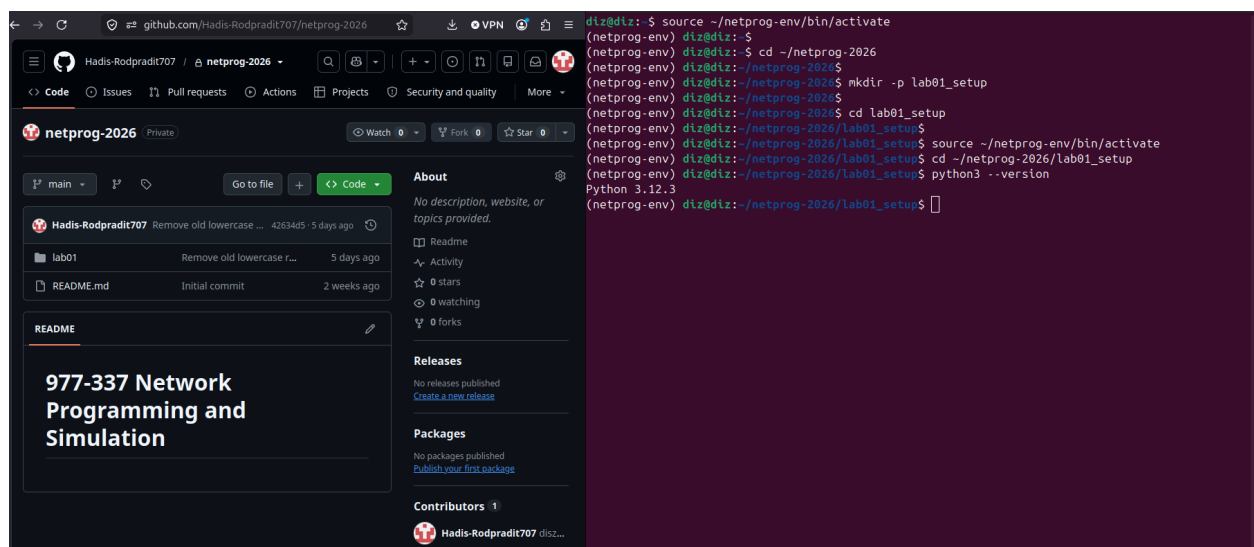


Figure 1 — The (netprog-env) prompt is active and python3 --version reports Python 3.12.3, with the netprog-2026 repository visible on GitHub.

Task 2 (Part B) — Hostname resolution & port classification

```
(netprog-env) diz@diz:~/netprog-2026/lab01_setup$ python3 port_lookup.py
Hostname: www.python.org
Port: 80
www.python.org resolves to 151.101.128.223
Port 80 is classified as: Well-Known
(netprog-env) diz@diz:~/netprog-2026/lab01_setup$ python3 port_lookup.py
Hostname: www.python.org
Port: 8080
www.python.org resolves to 151.101.128.223
Port 8080 is classified as: Registered
(netprog-env) diz@diz:~/netprog-2026/lab01_setup$ python3 port_lookup.py
Hostname: www.python.org
Port: 60000
www.python.org resolves to 151.101.128.223
Port 60000 is classified as: Dynamic/Private
(netprog-env) diz@diz:~/netprog-2026/lab01_setup$
```

Figure 2 — port_lookup.py resolves www.python.org and correctly classifies one port from each range: 80 (Well-Known), 8080 (Registered), 60000 (Dynamic/Private).

Task 3 (Part C) — TCP three-way handshake (ESTABLISHED)

```
diz@diz:~/netprog-2026/lab01_setup$ ss -tn | grep 151.101
ESTAB 0      0      172.19.185.191:54018    151.101.64.223:443
diz@diz:~/netprog-2026/lab01_setup$
```

Figure 3 — While `connect_demo.py` held the connection open, `ss -tn` showed an ESTABLISHED TCP connection to 151.101.64.223:443 (www.python.org), confirming the completed SYN / SYN-ACK / ACK handshake.

Task 4 (Part D) — Byte order (endianness)

```
(netprog-env) diz@diz:~/netprog-2026/lab01_setup$ python3 byteorder_demo.py
This machine's native byte order: little
Big-endian bytes:      12345678
Little-endian bytes:  78563412
Port 8080 in network byte order (htons): 36895
Packed with struct ('!H' = network-order unsigned short): b'\x1f\x90'
```

Figure 4 — `byteorder_demo.py` prints the native order (little), the big-endian and little-endian byte sequences, and the network-byte-order conversion via `htons` / `struct`.

Explanation. The big-endian and little-endian hex strings (12345678 vs 78563412) contain the same four bytes but in reverse order. They represent the same value (0x12345678) because endianness only changes the order the bytes are stored in memory, not the number itself—big-endian stores the most-significant byte first, little-endian stores the least-significant byte first.

Task 5 (Part E) — Manual HTTP request-response

```
(netprog-env) diz@diz:~/netprog-2026/lab01_setup$ python3 http_probe.py
HTTP/1.1 200 OK
Date: Mon, 31 Aug 2026 07:10:55 GMT
Content-Type: text/html
Transfer-Encoding: chunked
Connection: close
Server: cloudflare
Last-Modified: Sun, 30 Aug 2026 04:11:49 GMT
Allow: GET, HEAD
Accept-Ranges: bytes
Age: 2857
cf-cache-status: HIT
CF-RAY: a33a37db1ceea1b7-BKK

22f
<!doctype html><html lang="en"><head><title>Example Domain</title><link rel="icon" href="data:,"><meta name="viewport" content="width=device-width, initial-scale=1"><style>body{background-color:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><body><div><h1>Example Domain</h1><p>This domain is for use in documentation examples without needing permission. Avoid use in operations.</p><p><a href="https://iana.org/domains/example">Learn more</a></p></div></body></html>

0
```

Figure 5 — `http_probe.py` sends a raw GET request over a TCP socket and receives a complete response from `example.com`: the status line HTTP/1.1 200 OK, response headers, and the HTML body.

Task 6.1 (Part F) — Fast failure: ConnectionRefusedError

```
(netprog-env) diz@diz:~/netprog-2026/lab01_setup$ python3 connect_demo.py
Traceback (most recent call last):
  File "/home/diz/netprog-2026/lab01_setup/connect_demo.py", line 2, in <module>
    s = socket.create_connection(("127.0.0.1", 59999))
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/socket.py", line 852, in create_connection
    raise exceptions[0]
  File "/usr/lib/python3.12/socket.py", line 837, in create_connection
    sock.connect(sa)
ConnectionRefusedError: [Errno 111] Connection refused
```

Figure 6.1 — Connecting to 127.0.0.1:59999, where nothing is listening, fails immediately with `ConnectionRefusedError` [Errno 111]: the OS rejects the connection at once.

Task 6.2 (Part F) — Blocking with no timeout (long hang)

```
(netprog-env) diz@diz:~/netprog-2026/lab01_setup$ python3 connect_demo.py
^CTraceback (most recent call last):
  File "/home/diz/netprog-2026/lab01_setup/connect_demo.py", line 2, in <module>
    s = socket.create_connection(("198.18.0.1", 80))
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/socket.py", line 837, in create_connection
    sock.connect(sa)
KeyboardInterrupt
```

Figure 6.2 — Connecting to an unreachable address with no timeout blocks indefinitely; the call hangs inside `sock.connect()` until interrupted manually with `Ctrl+C` (`KeyboardInterrupt`).

Task 6.3 (Part F) — `settimeout(3)`: socket timeout

```
(netprog-env) diz@diz:~/netprog-2026/lab01_setup$ python3 connect_demo.py
Traceback (most recent call last):
  File "/home/diz/netprog-2026/lab01_setup/connect_demo.py", line 4, in <module>
    s.connect(("198.18.0.1", 80))
TimeoutError: timed out
```

Figure 6.3 — After adding `settimeout(3)`, the same unreachable connection raises `TimeoutError` after about three seconds instead of hanging—the timeout bounds the blocking call.

Verification Checklist

#	Checkpoint	Status
1	netprog-env virtual environment created and activates without errors	Done
2	netprog-2026 repository created on GitHub and cloned locally	Done
3	port_lookup.py correctly resolves a hostname and classifies all three port ranges	Done
4	An ESTABLISHED TCP connection to port 443 was observed while connect_demo.py was running	Done
5	byteorder_demo.py prints different hex byte sequences for big-endian vs little-endian	Done
6	http_probe.py prints a complete HTTP response (status line + headers) from a real server	Done
7	Removing the timeout causes a long hang; adding <code>settimeout(3)</code> raises <code>socket.timeout</code> instead	Done

Reflection Questions

1. Explain blocking I/O in the context of `connect()` and `recv()`. What is your program (and the underlying thread) doing, from the OS’s point of view, while it waits in Part F?

Blocking means the call does not return until the operation finishes. When `connect()` or `recv()` blocks, my program’s thread is put to sleep by the OS and taken off the CPU—it is not spinning or using CPU time, it is simply waiting in a blocked state. The OS parks the thread until an event wakes it: the TCP handshake completing, data arriving, an error such as connection-refused, or a timeout expiring. In Part F, while `connect()` waited on the unreachable address, the thread stayed blocked doing nothing until either I interrupted it (`Ctrl+C`) or the `settimeout(3)` deadline fired.

2. Why does `socket.htons()` exist at all—why can’t every machine just send integers in its own native byte order?

Different CPUs store multi-byte integers in different byte orders (little-endian vs big-endian), so if each machine sent integers in its own native order, a value like a port number would be read incorrectly by a machine with the opposite order. To avoid this, the network defines one common order—big-endian, or “network byte order.” `socket.htons()` converts a value from host order to network order before it goes on

the wire, so both ends agree on how to interpret the bytes regardless of their CPU architecture.

3. In Part E you built the “request” half of a request-response exchange by hand. Identify one thing a real HTTP client library (like Python’s requests) handles for you that you had to do manually or ignored here.

In `http_probe.py` I had to build the raw request text and read the reply as raw bytes, without interpreting it. A library like `requests` parses the response for me—it separates the status line, headers and body, decodes the Transfer-Encoding: chunked framing (visible as the “22f ... 0” chunk markers in my output) into the actual body, and applies the correct character encoding. It also manages the connection, follows redirects, and sets required headers automatically, none of which my raw-socket version handled.